

GLANCE INTERNSHIP ASSIGNMENT

Intelligent Fashion Search Engine with Structured Attribute-Augmented Dual-Retrieval

Candidate Name:	Aayush Kumbharkar
Email:	aayushkumbharkar53@gmail.com
GitHub Repository:	https://github.com/aayushkumbharkar/glance-fashion-retrieval
OS/Environment:	Windows / Python 3.10 / Local Venv

This submission satisfies both Part A (Indexer) and Part B (Retriever) workflows, resolving fine-grained clothing attribute mappings, setting extraction, and compositional color query binding.

1. Alternative Approaches and Tradeoffs

To map natural language text queries directly to clothing images, several visual and multimodal architectures were evaluated. Below are the three core design pathways analyzed for this project:

Approach	Multimodal Alignment	Core Weakness	Optimal Use Case
1. Vanilla CLIP Only (Contrastive Visual-Text)	Dense cosine alignment of images and textual query vectors in a single shared representation space.	Compositional Failure: Conflates color-garment bounds (e.g. 'red tie, white shirt' vs. 'white tie, red shirt'). Suffers from fashion vocabulary gap.	General visual scene search or broad palette-matching where exact bounding and grammatical detail are out of scope.
2. Pure Text Search (VLM + BM25/Dense)	Run local VLM on images. Index descriptions in a text engine (like Elasticsearch or a dense sentence embedder).	Lack of Visual Texture: Fails to capture non-verbal details, textures, or exact visual style attributes omitted by VLM captions.	Strict attribute matching where exact text descriptions or SKU codes govern searches.
3. Hybrid Dual-Retrieval (Structured + Visual)	Two-path search combining OpenCLIP visual similarity, BGE description matching, and exact categorical metadata bonuses.	Slight Indexing Overhead: Requires running a local VLM once at ingest time. Retrieval latency remains sub-100ms.	Chosen Solution: High-precision fashion query search requiring exact color-garment bounds and visual vibe preservation.

2. Short Write-up on Chosen Approach

Our chosen approach, **Structured Attribute-Augmented Dual-Retrieval**, is explicitly engineered to address CLIP's compositionality failures in fashion query processing. The workflow is divided into two distinct components:

The Indexer Workflow (Part A):

- **VLM Captions (Designed Path — BLIP-2):** The primary indexing path runs each image through **Salesforce/blip2-flan-t5-xl**, prompted to output a structured JSON object identifying clothing items, accessories, bound color pairs (e.g. 'navy jacket'), setting, and style. This path is fully implemented in *caption_generator.py* and is activated by running the indexer *without* the `--use_lightweight_vlm` flag. It requires a CUDA GPU (~8 GB VRAM; supported with 4-bit quantization at ~6 GB).
- **Submitted Index — CPU Fallback Path (Documented Tradeoff):** Due to no local GPU access within the 4-hour time budget, this submission's index was built using the **LightweightCaptionGenerator** (base BLIP, free-text caption + heuristic keyword-proximity parsing). Real numbers from *data/captions.json* (1,000 images): `parse_tier_used=3` for 100.0% of images; `environment=unknown` for 57.5%; `style=unknown` for 91.8%. This is a one-line CLI flag change: dropping `--use_lightweight_vlm` and running on a Colab T4 GPU would populate the full

structured schema at scale. The assignment's design, code, and architecture are built for the BLIP-2 path; only the submitted data artifact reflects the fallback.

- **3-Tier Parser Safety Net:** If the VLM output fails to parse as clean JSON (tier 1), the indexer falls back to regex extraction (tier 2) and then a proximity keyword search (tier 3) — capturing nearby colors for every garment found — to preserve metadata integrity.
- **Dual Fingerprints:** We save two mathematical vectors (embeddings): **CLIP Visual Features** (capturing palette, style, and visual layout) and **BGE Text Features** (embedding the structured caption prose). Storing both in separate **ChromaDB** drawers linked by a shared ID enables multi-angle querying.

The Retriever Workflow (Part B):

- **Query Decomposition & Expansion:** An LLM (Llama-3.1-8b via Groq) performs zero-shot query decomposition to map free-form search text to our caption schema (capturing clothing_items, colors, environment, style). Crucially, colors are bound to specific items (e.g. 'red tie', 'white shirt') to resolve compositionality, and the query is expanded into descriptive prose to bridge the domain gap.
- **Hybrid Fusion Scoring:** We search both the CLIP and BGE collections to retrieve candidate pools of size 3k. Similarity is computed as cosine similarity ($\text{sim} = 1 - \text{distance}$) on L2-normalized embeddings. The final query scoring applies a weighted sum: **Score** = $\alpha \cdot \text{Sim_CLIP} + \beta \cdot \text{Sim_BGE} + \text{Bonus}$, where the default weights are $\alpha = 0.35$ (CLIP visual vibe & scene layout) and $\beta = 0.50$ (BGE compositional description matching). $\beta > \alpha$ because structured text handles word order and binding far better than a pure contrastive visual space.
- **Categorical Match Bonus:** The third term represents a hard metadata matching bonus (**Bonus** $\leq \gamma = 0.15$) calculated by comparing parsed query attributes against the image metadata. Matching environments (e.g., 'office') yield +40% of γ , matching style yields +30%, while bound color-garment pairs yield +10% each. This rewards exact matches, pushing true-positives above false-positives with high soft vector similarity.

3. Codebase Structure & GitHub Link

The complete codebase is organized to keep data logic separate from models, making it highly modular and scalable.

Official Repository: <https://github.com/aayushkumbharkar/glance-fashion-retrieval>

Directory / File	Function & Responsibility
<code>`data/download_dataset.py`</code>	Streams Fashionpedia zip archives directly from AWS S3, extracting exactly 1,000 images sequentially and saving the manifest locally.
<code>`Part_A_Indexer/`</code>	Contains VLM captioning (<code>`caption_generator.py`</code>), dual visual/text feature embedding (<code>`feature_extractor.py`</code>), and ChromaDB collection management (<code>`vector_store.py`</code>).
<code>`Part_B_Retriever/`</code>	Contains Groq LLM query parsing (<code>`query_parser.py`</code>), hybrid fusion scoring with metadata bonuses (<code>`retriever.py`</code>), and the FastAPI server + Web search dashboard (<code>`run_retrieval.py`</code>).
<code>`evaluation/run_eval_queries.py`</code>	Runs all 5 evaluation queries from the assignment brief, computes latency, and saves the output locally as a JSON report.
<code>`notebooks/exploration.ipynb`</code>	Contains PCA/UMAP embedding projections, EDA distribution charts, and configurations sensitivity sweeps.

4. Future Work Roadmap

A. Location & Weather Awareness Extension

To incorporate local context like weather and cities without changing the underlying vector space structure, we can leverage EXIF metadata and classification heads:

- **Location:** Extract GPS metadata from image headers at index time. Use an offline reverse geocoding API (e.g. OpenStreetMap's Nominatim) to convert coordinates to categorical place names (e.g., 'San Francisco', 'Coit Tower', 'coastal city'). Add these as keywords to the text embedding and metadata filter records.
- **Weather:** Fine-tune a lightweight visual classifier (like a ViT) on public weather datasets to tag images into standard classes (Sunny, Rainy, Snowy, Overcast). The parsed environment query (e.g., 'city walk on a rainy day') can then trigger strict metadata filters (``weather: rainy``), preventing sunny outfits from polluting the results.

B. Improving Search Precision

To move from zero-shot baselines to a production-grade visual fashion engine, four precision pipelines would be implemented:

- **Caption Quality (Highest-Leverage Next Step):** The current submission's caption quality is bottlenecked by CPU-only heuristic parsing, evidenced by 57.5% environment=unknown and 91.8% style=unknown rates (see

evaluation/results.json). Running the BLIP-2 structured path (already implemented, gated behind a CLI flag: `drop --use_lightweight_vlm`) on a Colab T4 GPU would directly resolve this and is the single highest-leverage next step for improving retrieval precision on environment- and style-filtered queries.

- **Fine-Tuning Visual Alignments:** Fine-tune OpenCLIP on domain-specific triplet datasets (like DeepFashion and FashionIQ) to close the fashion vocabulary gap (e.g. distinguishing a 'parka' from an 'anorak').
- **Cross-Encoder Re-ranking:** Retrieve the top 50 candidates using our dual-space index, then feed (query, caption text) pairs into a BERT-based Cross-Encoder. Because Cross-Encoders evaluate full attention over both texts together, they identify subtle mismatches that bi-encoder search misses.
- **Region-Based Feature Maps:** Segment each image into visual crops (upper-body, lower-body, accessories) using segmenters like SAM. Generate separate CLIP vectors per crop. When a query searches for 'red tie and white shirt', we match the tie embedding against the accessory/upper crops specifically, resolving compositional binding visually.
- **Click-Through Bandit Loop:** Track user click-through rates on search results. Use contextual bandits to auto-tune alpha, beta, and gamma weights dynamically, adapting ranking scores to user intent.